

3.5.3 The while Loop, break and continue

while لوپ اس وقت تک چلتا رہتا ہے جب تک شرط درست (True) رہے، اور یہ تب استعمال ہوتا ہے جب پہلے سے معلوم نہ ہو ...

CS XI

Chapter 3: Programming Fundamentals

Lecture 18 of 24

★ Golden

3.5.3 The while Loop, break and continue ★ Golden

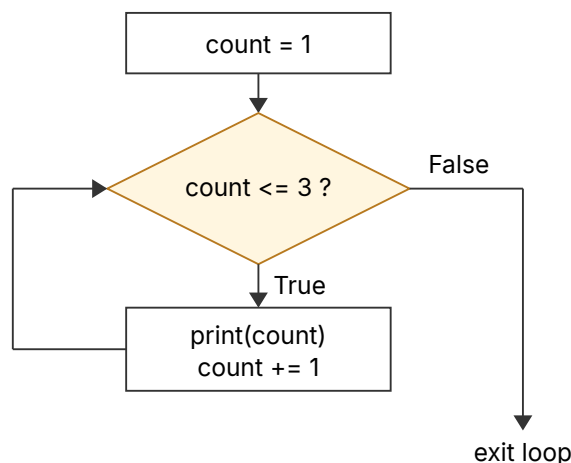
LEARN IT

2. The **while loop** repeats a block of code **as long as a condition remains True**. It is used when the exact number of iterations is **not known in advance** (for example, "keep asking until the password is correct").

A while loop needs three parts: (1) **initialise** a variable before the loop, (2) **test** the condition, (3) **update** the variable inside the loop. If you forget the update, the condition never becomes False and the loop never stops — an **infinite loop**.

Loop control statements change the normal flow of a loop:

- **break** — exits (stops) the loop **entirely and immediately**.
- **continue** — **skips the rest of the current iteration** and jumps to the next one.



A while loop: test the condition, run the body, go back and test again.

```

count = 1
while count ≤ 3:
    print(count)
    count += 1  # VERY IMPORTANT: update the condition!
  
```

OUTPUT

1
2
3

WORKED EXAMPLE

Trace of the while loop above:

count	count <= 3?	Printed	count after update
1	True	1	2
2	True	2	3
3	True	3	4
4	False	—	loop ends

Sum of digits — we do not know how many digits the number has, so `while` fits well:

```
n = 4072
total = 0
while n > 0:
    digit = n % 10
    total += digit
    n = n // 10
print("Sum of digits =", total)
```

```
OUTPUT
Sum of digits = 13
```

Digits taken: 2, 7, 0, 4 → 2 + 7 + 0 + 4 = 13. `n` becomes 407, 40, 4, 0, and then the loop stops.

Password check — repeat until the user gets it right:

```
password = input("Enter password: ")
while password != "sindh123":
    print("Wrong password, try again.")
    password = input("Enter password: ")
print("Welcome!")
```

Sample run (user types `abc`, `sindh`, `sindh123`):

```
OUTPUT
Enter password: abc
Wrong password, try again.
Enter password: sindh
Wrong password, try again.
Enter password: sindh123
Welcome!
```

break and continue side by side:

```
for i in range(5):
    if i == 3:
        break
    print(i)
print("Loop ended")
```

```
OUTPUT
0
1
2
Loop ended
```

```
for i in range(5):
    if i == 3:
        continue
    print(i)
```

```
OUTPUT
0
1
2
4
```

At `i = 3` the loop stops completely, so 3 and 4 are never printed.

At `i = 3` only that iteration is skipped; the loop carries on with 4.

while True with break — a loop that runs "forever" until `break` stops it:

```
n = 1
while True:
    if n * n > 50:
        break
    n += 1
print("First number whose square is more than 50:", n)
```

```
OUTPUT
First number whose square is more than 50: 8
```

REMEMBER IT

break = "exit the room"; **continue** = "skip this turn". For every `while` loop, check the **I-T-U** rule: Initialise, Test, Update.

COMMON MISTAKE

Infinite loop — find the error:

```
count = 1
while count ≤ 5:
    print(count)
```

`count` is never updated, so `count ≤ 5` stays True and the program prints 1 forever. Fix: add `count += 1` inside the loop. (To stop a running program in the terminal, press **Ctrl + C**.)

اردو میں سمجھیں

`while` لوپ اس وقت تک چلتا رہتا ہے جب تک شرط درست (True) رہے، اور یہ تب استعمال ہوتا ہے جب پہلے سے معلوم نہ ہو کہ کتنی بار دہرانا ہے۔ لوپ کے اندر ویری ایبل کو اپڈیٹ کرنا لازمی ہے ورنہ لوپ کبھی ختم نہیں ہوگا (Infinite Loop)۔ `break` لوپ کو فوراً مکمل طور پر روک دیتا ہے، جبکہ `continue` صرف موجودہ چکر چھوڑ کر اگلے چکر پر چلا جاتا ہے۔

EXAM TIP

"Differentiate `break` and `continue` with examples" and "What is an infinite loop?" are frequent. Use the two side-by-side programs above with their outputs — examiners like seeing the difference in the output (0 1 2 vs 0 1 2 4).

CHECK YOURSELF

1. Predict the output:

```
n = 1
while n < 20:
    n = n * 3
    print(n)
```

2. Predict the output:

```
for i in range(1, 8):
    if i % 3 == 0:
        continue
    if i == 7:
        break
    print(i, end=" ")
```

Answers: 1. 27 (`n`: 1 → 3 → 9 → 27; 27 < 20 is False, so the loop stops). 2. 1 2 4 5 (3 and 6 are skipped by `continue`; at 7 the loop breaks).

Next lecture: 3.5.3 Nested Loops and `for` vs `while`